Header file code is: Book.h

```
#pragma once

#include<iostream>
#include<fstream>
#include<functional>
#include<string>

class Book
{
private:
	std::string author, name, specialty;
	int year, pages;
 int count;
public:
	Book() {};
	Book(std::string author, std::string name, int years, int pages_, std::string
specialty) :
		author(author), name(name), year(years), pages(pages_),
specialty(specialty) {
		count += 1;
	};

	std::string get_author() { return author; };
	std::string get_name() { return name; };
	std::string get_specialty() { return specialty; };
	int get_year() { return year; };
	int get_count_of_pages() { return pages; };
	int get_count() { return count; }

	void set_author(std::string s) { author = s; };
	void set_name(std::string s) { name = s; };
	void set_specialty(std::string s) { specialty = s; };
	void set_years(int years) { year = years; };
	void set_pages(int page) { pages = page; };

	bool operator==(Book other) { return year == other.get_year(); };
	bool operator!=(Book other) { return year != other.get_year(); };
	bool operator>(Book other) { return year > other.get_year(); };
	bool operator>=(Book other) { return year >= other.get_year(); };
	bool operator<(Book other) { return year < other.get_year(); };
	bool operator<=(Book other) { return year <= other.get_year(); };

	friend std::istream& operator>>(std::istream& in, Book& book);
	friend std::ostream& operator<<(std::ostream& out, Book& book);


};
```

The code for the Book.cpp is:

```
#include "Book.h"

std::istream& operator>>(std::istream& in, Book& book)
{
    std::string s;
```

```
    int value;

    std::getline(in, s);
    book.set_author(s);

    std::getline(in, s);
    book.set_name(s);

    in >> value;
    book.set_years(value);

    in >> value;
    book.set_pages(value);
    in.ignore();

    std::getline(in, s);
    book.set_specialty(s);

    in.ignore();
    book.count += 1;
    return in;
}

std::ostream& operator<<(std::ostream& out, Book& book)
{
    out << book.author << '\n'
        << book.name << '\n'
        << book.year << '\n'
        << book.pages << '\n'
        << book.specialty << '\n';
    return out;
}
```

Header (Flist.h) code is:

```
#pragma once

#include"Book.h"

using TInfo = Book;

class NODE
{
public:
      TInfo info;
      NODE* next;
      NODE() {}
      NODE(TInfo info, NODE* ptr = nullptr) : info(info), next(ptr) {}
      ~NODE() { next = nullptr; }
};

using ptrNODE = NODE*;
class FList
{
private:
```

```
        ptrNODE head, tail;
        void add_by_pointer(ptrNODE& ptr, TInfo value);
public:
        FList() { head = tail = nullptr; }
        FList(std::ifstream& file);
        FList(const FList& other);
        FList& operator=(const FList& other);
        FList(FList&& tmp);
        FList& operator=(FList&& tmp);
        ~FList();

        ptrNODE get_head() const { return head; }
        bool empty();
        void add_to_head(TInfo value);
        void add_to_tail(TInfo value);
        void print();
        void del_from_head();
        void clear();
};
```

Flist.cpp code is:

```
#include "Flist.h"

void FList::add_by_pointer(ptrNODE& ptr, TInfo value)
{
    ptr = new NODE(value, ptr);
}

FList::FList(std::ifstream& file)
{
    TInfo value;
    tail = head = nullptr;
    auto insert = [this](TInfo value) -> ptrNODE
        {
            ptrNODE result = head;
            while (result->next && result->next->info > value)
                result = result->next;
            return result;
        };

    while (file >> value)
    {
        if (empty() || head->info <= value)
            add_to_head(value);
        else
            add_by_pointer(insert(value)->next, value);
    }
}

FList::FList(const FList& other)
{
    ptrNODE ptr = other.head;
    this->head = this->tail = nullptr;
    while (ptr)
```

```
    {
        this->add_to_tail(ptr->info);
        ptr = ptr->next;
    }
}

FList& FList::operator=(const FList& other)
{
    if (this != &other)
    {
        clear();
        ptrNODE ptr = other.head;
        while (ptr)
        {
            this->add_to_tail(ptr->info);
            ptr = ptr->next;
        }
    }
    return *this;
}

FList::FList(FList&& tmp)
{
    this->head = tmp.head;
    this->tail = tmp.tail;
    tmp.tail = tmp.head = nullptr;
}

FList& FList::operator=(FList&& tmp)
{
    if (this != &tmp)
    {
        this->clear();
        this->head = tmp.head;
        this->tail = tmp.tail;
        tmp.head = nullptr;
        tmp.tail = nullptr;
    }
    return *this;
}

FList::~FList()
{
    clear();
}

bool FList::empty()
{
    return !head;
}

void FList::add_to_head(TInfo value)
{
    add_by_pointer(head, value);
}

void FList::add_to_tail(TInfo value)
{
```

```
    if (tail)
    {
        add_by_pointer(tail->next, value);
        tail = tail->next;
    }
    else
        add_to_head(value);
}

void FList::print()
{
    ptrNODE ptr = head;
    while (ptr)
    {
        std::cout << ptr->info << "--------------------\n";
        ptr = ptr->next;
    }
}

void FList::del_from_head()
{
    ptrNODE p = head;
    head = head->next;
}

void FList::clear()
{
    while (head)
        del_from_head();
    head = tail = nullptr;
}
```

Source.cpp file code is:

```
#include"Flist.h"
#include<Windows.h>

void my_task(const FList& list, int year)
{
	FList tmp;
	ptrNODE head = tmp.get_head();
	while (head && head->info.get_year() < year) {
		ptrNODE temp = tmp.get_head();
		head = head->next;
		delete temp;
	}
	ptrNODE current = list.get_head();
	while (current->next && current->next->info.get_year() < year) {
		ptrNODE temp = current->next;
		current->next = current->next->next;
		delete temp;
	}
}

FList remove_books_By_Year(const FList& list, int year) {
```

```
	FList newList;
	ptrNODE ptr = list.get_head();

	while (ptr) {
		if (ptr->info.get_year() >= year)
		{
			newList.add_to_tail(ptr->info);
		}
		ptr = ptr->next;
	}

	return newList;
}

int main()
{
	SetConsoleOutputCP(1251);
	std::ifstream file("file.txt");
	if (file)
	{
		FList list(file);
		list.print();
		std::cout << "************************** My Task
**********************************\n";
		int year;
		std::cout << "Enter the Year: ";
		std::cin >> year;

		FList result = remove_books_By_Year(list, year);
		std::cout << "\n\n";
		result.print();

		//my_task(list, year);
		//list.print();

		//std::cout << "Total number of books: " << Book::get_count() <<
std::endl;

		/*std::cout << "----------------\n";
		FList a(list);
		a = result;
		FList b(std::move(list));
		b = std::move(result);*/


	}
	else
		std::cout << "Empty file!\n";

	return 0;
}
```

File data is:

```
John Smith
```

The Art of Programming
1998
500
Computer Science
Emily Johnson
Introduction to Machine Learning
2010
350
Artificial Intelligence
Michael Brown
Data Structures and Algorithms
2005
600
Computer Science
Sarah Williams
History of Philosophy
1995
400
Philosophy
David Miller
Fundamentals of Physics
2015
800
Physics
Jessica Davis
Modern Art: A Comprehensive Guide
2008
450
Art
Matthew Taylor
The History of Mathematics
2012
700
Mathematics
Amanda White
The Great Novels of Russian Literature
2003
550
Literature
Christopher Wilson
Chemistry: Principles and Applications
2017
750
Chemistry
Jennifer Martinez
Introduction to Economics
2006
400
Economics
Daniel Thompson
The History of World War II
1990
900
History
Laura Anderson
Introduction to Psychology
2019
600

Psychology
Kevin Thomas
Programming in Python
2014
350
Computer Science
Rachel Harris
Essentials of Biology
2011
550
Biology
Samuel Moore
The Art of Negotiation
2009
400
Business
